

GANDAKI COLLEGE OF ENGINEERING AND SCIENCE

Lamachaur, 18, Kaski, Nepal



LAB REPORT OF Agile software development

**Lab2: Implementation of source code management and version control with
git and github**

Submitted By:	Submitted To:
Sandip Aryal Roll no: 35	Er.Rajindra Bdr. Thapa

1. Objective

The primary objective of this lab is to explore and implement the core features of the Visual Studio Code (VS Code) Integrated Development Environment (IDE).

2. Tools and Technologies

Visual Studio Code: A lightweight, open-source code editor developed by Microsoft, equipped with robust built-in IDE features.

- **VS Code Extensions:** Add-on packages (e.g., Prettier, Live Server, Python) that enhance the editor's core capabilities.
- **Integrated Terminal:** A command-line interface embedded directly within the VS Code workspace.
- **Git (via VS Code Source Control):** Built-in interface for version control operations.

3. Theory

An Integrated Development Environment (IDE) is a software suite that consolidates basic developer tools into a single graphical user interface (GUI). While traditional text editors merely allow for writing code, modern IDEs provide essential features such as syntax highlighting, auto-completion (IntelliSense), code refactoring, integrated debugging, and version control, all in one cohesive environment.

Visual Studio Code has become the industry standard for many developers because it strikes an optimal balance between a lightweight editor and a full-fledged IDE. Instead of being a monolithic, resource-heavy application, VS Code is highly modular. Its power largely comes from an extensive marketplace of extensions, allowing developers to tailor the environment perfectly for any specific programming language (e.g., Python, JavaScript, C++) or framework (e.g., React, Django, Node.js).

This lab demonstrates the fundamental features of VS Code, highlighting how they work together to reduce context switching, minimize errors, and accelerate the iterative development process essential for agile methodologies.

4. Implementation Steps and Observations

The following steps outline the workflow for setting up VS Code and utilizing its key features to manage a simple coding project.

1. Initial Setup and Customization

After downloading and installing VS Code from the official website, the initial configurations must be adjusted to suit individual preferences.

- **Opening a Workspace:** Created a new project folder and opened it via File > Open Folder.
- **Theme and Appearance:** Selected a color theme via File > Preferences > Color Theme (e.g., Dark+ or Light+).
- **Basic Settings:** Accessed settings using `Ctrl + ,` (or `Cmd + ,` on Mac). Enabled Editor: Auto Save and adjusted Editor: Font Size.

2. Mastering the Command Palette

The Command Palette is the core command center of VS Code, allowing access to almost every function without needing to memorize complex menu navigation.

- Opened the Command Palette using `Ctrl + Shift + P`.
- Typed `>` to view all available commands.
- Executed commands like `> Go to File` to quickly switch between documents, and `> Format Document` to quickly apply code formatting.

3. Extending Functionality with Extensions

Extensions are the backbone of VS Code's versatility, enabling support for virtually any programming language or development tool.

- Clicked the Extensions view icon on the left activity bar.
- Searched for the "Prettier - Code formatter" extension.
- Clicked **Install** to add it to the workspace.
- Created a script.js file with intentionally messy indentation. Used the Command Palette to run `> Format Document`, which automatically cleaned up the spacing and syntax using Prettier rules.

4. Utilizing the Integrated Terminal

The integrated terminal provides a command-line environment directly inside the editor, eliminating the need to switch between the IDE and a separate external window.

- Opened the terminal using Ctrl + (backtick) or via View > Terminal`.
- Navigated to the current project directory manually (though it usually opens in the project root automatically).
- Entered basic shell commands, such as mkdir test-folder and echo "console.log('Hello World');" > script.js to create new files directly from the terminal.

5. Source Control and Git Integration

VS Code includes a deep integration with Git, providing a visual interface for version control operations.

- Opened the Source Control view using the left-side icon.
- Clicked Initialize Repository to create a local Git repository for the current project.
- Modified script.js. The Source Control panel immediately highlighted the changed lines with a visual "diff" (differences viewer).
- Staged the file using the + button, typed a commit message, and committed the change using the checkmark icon.

6. Integrated Debugging

VS Code features a powerful built-in debugger that allows developers to pause code execution and inspect variables in real-time.

- Created a simple JavaScript file with a loop to iterate over an array.
- Set a **breakpoint** by clicking to the left of the line number on the desired code line.
- Clicked the Run and Debug tab on the left, then selected the Run button.
- The code execution paused at the breakpoint. Used the Debug toolbar (Step Over, Step Into, Step Out) to move through the code line by line, observing variable values in the VARIABLES pane.

5. Conclusion

This lab successfully demonstrated the core features that make Visual Studio Code a premier choice for modern developers. By exploring the Command Palette, the extension marketplace, the integrated terminal, version control, and debugging capabilities, we can conclude that VS Code provides a unified development ecosystem. Mastering these tools not only reduces friction in daily coding tasks but also aligns perfectly with the demands of Agile development enabling rapid prototyping, frequent testing, and seamless collaboration through its modular and extensible design.

